ThreatMatrix AI — Master Documentation v1.0

Part 2: System Architecture & Infrastructure Blueprint

> **Part:** 2 of 5 | **Version:** 1.0.0 | **Date:** 2026-02-23
> **Prev:** Part 1 — Strategy | **Next:** Part 3 — Modules

Table of Contents — Part 2

1. Architecture Overview
2. Three-Tier Architecture Deep Dive
3. Infrastructure & Deployment Topology
4. Database Schema Design
5. API Architecture & Design
6. Real-Time Communication Strategy
7. Security Architecture
8. Technology Stack Decisions
9. Cost Infrastructure Analysis

---

1. Architecture Overview

1.1 Architecture Philosophy

ThreatMatrix AI follows a **pragmatic enterprise architecture** — impressive enough to demonstrate distributed systems knowledge, but buildable within 8 weeks by a lean team. Every component serves a purpose; nothing is included for résumé-driven development.

1.2 High-Level System Diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        THREATMATRIX AI ARCHITECTURE v1.0                     │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│   ┌──────────────────────────────────────────────────────────────────────┐   │
│   │              TIER 1: CAPTURE ENGINE (Python / Scapy)                 │   │
│   │  • Live packet capture on VPS interfaces                             │   │
│   │  • NetFlow-style flow aggregation (5-tuple + timing + volume)        │   │
│   │  • Feature extraction pipeline (40+ features per flow)               │   │
│   │  • PCAP file ingestion for historical/forensic analysis              │   │
│   │  • Publishes flow data to Redis Pub/Sub                              │   │
│   └──────────────────────────────────┬───────────────────────────────────┘   │
│                                      │ Redis Pub/Sub + REST                  │
│   ┌──────────────────────────────────▼───────────────────────────────────┐   │
│   │              TIER 2: INTELLIGENCE ENGINE (Python / FastAPI)          │   │
│   │                                                                      │   │
│   │  ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐│   │
```

```
│   │   │ ML Inference │ │ Threat Intel │ │ LLM Gateway    │ │ Alert Mgmt   │ │   │
│   │   │ • IsoForest  │ │ • OTX Feed   │ │ • DeepSeek     │ │ • Scoring    │ │   │
│   │   │ • RandForest │ │ • AbuseIPDB  │ │ • GLM          │ │ • Lifecycle  │ │   │
│   │   │ • Autoencoder│ │ • VirusTotal │ │ • Groq         │ │ • Broadcast  │ │   │
│   │   └──────────────┘ └──────────────┘ └────────────────┘ └──────────────┘ │   │
│   │                                                                         │   │
│   │   ┌──────────────┐ ┌──────────────┐ ┌──────────────────────────────┐   │   │
│   │   │ PostgreSQL   │ │ Redis        │ │ Background Workers           │   │   │
│   │   │ • Flows      │ │ • Pub/Sub    │ │ • Feed sync (cron)           │   │   │
│   │   │ • Alerts     │ │ • Cache      │ │ • Model retraining           │   │   │
│   │   │ • Intel      │ │ • Sessions   │ │ • Report generation          │   │   │
│   │   │ • Users      │ │ • Rate limit │ │ • Cleanup jobs               │   │   │
│   │   └──────────────┘ └──────────────┘ └──────────────────────────────┘   │   │
│   └─────────────────────────────────────────────────────────────────────────┘   │
│                                   │                                             │
│                                   │ REST API + WebSocket                        │
│   ┌───────────────────────────────▼─────────────────────────────────────────┐   │
│   │              TIER 3: COMMAND CENTER (Next.js 16 / TypeScript)           │   │
│   │                                                                         │   │
│   │  10 Modules: War Room │ Threat Hunt │ Intel Hub │ Network Flow │      │   │
│   │  AI Analyst │ Alert Console │ Forensics Lab │ ML Ops │ Reports │ Admin│   │
│   │                                                                         │   │
│   │  Features: WebSocket live updates │ Deck.gl/Maplibre maps │            │   │
│   │  Glassmorphism UI │ Amharic/English i18n │ PDF export │ Dark theme    │   │
│   └─────────────────────────────────────────────────────────────────────────┘   │
│                                                                                 │
└─────────────────────────────────────────────────────────────────────────────────┘
```

---

2. Three-Tier Architecture Deep Dive

2.1 Tier 1: Capture Engine

Purpose: Intercept, parse, and transform raw network packets into structured flow records with ML-ready features.

Technology: Python 3.11+ with Scapy

Component	Responsibility	Implementation
Packet Sniffer	Raw packet capture on network interfaces	Scapy `sniff()` with BPF filters
Flow Aggregator	Group packets into bidirectional flows (5-tuple)	Custom aggregator with configurable timeout (30s/120s)
Feature Extractor	Compute 40+ features per flow for ML input	Duration, packet counts, byte stats, protocol flags, inter-arrival times
Protocol Analyzer	Deep inspection of DNS, HTTP, TLS, ICMP	Scapy dissection layers

Component	Responsibility	Implementation
PCAP Ingester	Read and process uploaded .pcap/.pcapng files	`rdpcap()` with chunked processing
Redis Publisher	Push completed flows to real-time pipeline	Redis Pub/Sub on `flows:live` channel

Flow Aggregation Logic:

```
Packet → 5-Tuple Key (src_ip, dst_ip, src_port, dst_port, protocol)
       → Accumulate in flow buffer
       → On timeout OR FIN/RST → Extract features → Publish → Store
```

Feature Vector (40+ features per flow):

Category	Features	Count
Basic	duration, protocol_type, service, flag	4
Volume	src_bytes, dst_bytes, total_bytes, byte_ratio	4
Packet	src_packets, dst_packets, total_packets, packet_ratio	4
Timing	mean_iat, std_iat, min_iat, max_iat (inter-arrival time)	4
TCP Flags	syn_count, ack_count, fin_count, rst_count, psh_count, urg_count	6
Connection	same_host_count, same_service_count, serror_rate, rerror_rate	4
DNS	query_count, response_count, unique_domains, avg_query_length	4
Payload	payload_entropy, mean_payload_size, has_payload	3
Behavioral	is_internal, port_class (well-known/registered/dynamic), geo_country	3
Statistical	flow_duration_zscore, byte_zscore, packet_zscore	3
Derived	packets_per_second, bytes_per_packet, connection_density	3+

2.2 Tier 2: Intelligence Engine

Purpose: Core brain of the system — ML inference, threat intel correlation, LLM analysis, alert management.

Technology: Python 3.11+ with FastAPI, PostgreSQL, Redis

Service Components

Service	Endpoint Prefix	Responsibility
Auth Service	`/api/v1/auth/`	JWT authentication, user management, RBAC
Flow Service	`/api/v1/flows/`	Flow data CRUD, search, aggregation

Service	Endpoint Prefix	Responsibility
ML Service	`/api/v1/ml/`	Model inference, scoring, model management
Alert Service	`/api/v1/alerts/`	Alert CRUD, lifecycle (open→ack→resolved), severity
Intel Service	`/api/v1/intel/`	Threat feed aggregation, IOC lookup, IP reputation
LLM Service	`/api/v1/llm/`	Chat interface, threat narratives, report generation
Capture Service	`/api/v1/capture/`	Capture engine control, PCAP upload, interface config
Report Service	`/api/v1/reports/`	PDF generation, scheduled reports
WebSocket	`/ws/`	Real-time event broadcasting
System Service	`/api/v1/system/`	Health checks, config, metrics

2.3 Tier 3: Command Center

Purpose: Intelligence agency-grade War Room interface for security analysts.

Technology: Next.js 16, TypeScript, Deck.gl, Maplibre GL, Recharts

Detailed in **Part 3** (Module Specifications).

---

3. Infrastructure & Deployment Topology

3.1 VPS Deployment Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│                      VPS (High-Spec Server)                          │
│                      Ubuntu 22.04 LTS                                │
├──────────────────────────────────────────────────────────────────────┤
│  Docker Compose Stack                                                │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────┐  │
│  │ PostgreSQL   │  │ Redis 7      │  │ FastAPI Backend          │  │
│  │ :5432        │  │ :6379        │  │ :8000                    │  │
│  │ Volume:      │  │ • Pub/Sub    │  │ • Uvicorn (4 workers)    │  │
│  │  pg_data     │  │ • LLM Cache  │  │ • WebSocket endpoint     │  │
│  └──────────────┘  │ • Rate Limit│  │ • OpenAPI auto-docs      │  │
│                    │ • Sessions   │  └──────────────────────────┘  │
│  ┌──────────────┐  └──────────────┘  ┌──────────────────────────┐  │
│  │ Capture      │                    │ ML Worker                │  │
│  │ Engine       │                    │ (Background Process)     │  │
│  │ • Host net   │                    │ • Inference loop         │  │
│  │ • Raw socket │                    │ • Model retraining cron  │  │
│  │ • Privileged │                    │ • Feed sync scheduler    │  │
│  └──────────────┘                    └──────────────────────────┘  │
│                                                                      │
│  Network: threatmatrix_net (bridge)                                  │
│  Capture: host network mode (for packet access)                      │
├──────────────────────────────────────────────────────────────────────┤
```

```
│  Nginx Reverse Proxy (:443 → :8000 backend API)                  │
│  SSL: Let's Encrypt via Certbot                                   │
└──────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────┐
│  Vercel (Frontend Hosting)                                        │
│  threatmatrix-ai.vercel.app                                       │
│  • Next.js 16 SSR/SSG                                             │
│  • API Routes proxy to VPS backend                                │
│  • Edge functions for auth middleware                              │
└──────────────────────────────────────────────────────────────────┘
```

3.2 Docker Compose Service Definitions

Service	Image	Ports	Volumes	Network Mode	Privileged
`postgres`	postgres:16-alpine	5432	pg_data	bridge	No
`redis`	redis:7-alpine	6379	redis_data	bridge	No
`backend`	threatmatrix/backend	8000	./models, ./logs	bridge	No
`capture`	threatmatrix/capture	—	./pcaps	**host**	**Yes**
`ml-worker`	threatmatrix/ml-worker	—	./models	bridge	No
`nginx`	nginx:alpine	80, 443	./certs, ./nginx.conf	bridge	No

3.3 Hybrid Capture Architecture

Primary mode: VPS traffic capture (legal, always available, real production traffic)

Optional: Local capture agent → sends flows to VPS backend via API

```
                    ┌──────────────┐
                    │   VPS        │
                   │ ┌──────────┐ │
┌─────────┐        │ │ FastAPI  │ │      ┌──────────────────┐
│ Local   │ HTTPS  │ │ Backend  │◄│──────│ VPS Capture      │
│ Agent   │──────► │ └──────────┘ │      │ Engine           │
│ (laptop)│        │              │      │ (host network)   │
└─────────┘        └──────────────┘      └──────────────────┘
 Optional            Central               Primary
 (demo any net)      (always on)            (production)
```

---

4. Database Schema Design

4.1 Schema Philosophy

Following the same **schema-first, DDD** approach proven in Pana ERP v3.0. Every table is designed before a single line of code is written.

4.2 Core Tables

Users & Auth

```
CREATE TABLE users (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email           VARCHAR(255) UNIQUE NOT NULL,
    password_hash   VARCHAR(255) NOT NULL,
    full_name       VARCHAR(255) NOT NULL,
    role            VARCHAR(50) NOT NULL DEFAULT 'viewer',  -- admin, soc_manager,
analyst, viewer
    language        VARCHAR(10) DEFAULT 'en',               -- en, am
    is_active       BOOLEAN DEFAULT true,
    last_login      TIMESTAMPTZ,
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    updated_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Network Flows

```
CREATE TABLE network_flows (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    timestamp       TIMESTAMPTZ NOT NULL,
    src_ip          INET NOT NULL,
    dst_ip          INET NOT NULL,
    src_port        INTEGER,
    dst_port        INTEGER,
    protocol        SMALLINT NOT NULL,           -- 6=TCP, 17=UDP, 1=ICMP
    duration        REAL,
    total_bytes     BIGINT,
    total_packets   INTEGER,
    src_bytes       BIGINT,
    dst_bytes       BIGINT,
    features        JSONB NOT NULL,              -- Full 40+ feature vector
    anomaly_score   REAL,                        -- ML-assigned score (0.0-1.0)
    is_anomaly      BOOLEAN DEFAULT false,
    ml_model        VARCHAR(50),                 -- which model classified
    label           VARCHAR(50),                 -- attack type if classified
    source          VARCHAR(20) DEFAULT 'live',  -- live, pcap, agent
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Performance indexes
CREATE INDEX idx_flows_timestamp ON network_flows(timestamp DESC);
CREATE INDEX idx_flows_src_ip ON network_flows(src_ip);
CREATE INDEX idx_flows_dst_ip ON network_flows(dst_ip);
CREATE INDEX idx_flows_anomaly ON network_flows(is_anomaly) WHERE is_anomaly =
true;
CREATE INDEX idx_flows_score ON network_flows(anomaly_score DESC);
```

Alerts

```
CREATE TABLE alerts (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    alert_id        VARCHAR(20) UNIQUE NOT NULL,  -- TM-ALERT-00001
    severity        VARCHAR(20) NOT NULL,          -- critical, high, medium, low,
info
    title           VARCHAR(500) NOT NULL,
    description     TEXT,
    category        VARCHAR(100),                  -- ddos, port_scan, c2,
dns_tunnel, brute_force
    source_ip       INET,
    dest_ip         INET,
    confidence      REAL,                          -- 0.0-1.0
    status          VARCHAR(20) DEFAULT 'open',    -- open, acknowledged,
investigating, resolved, false_positive
    assigned_to     UUID REFERENCES users(id),
    flow_ids        UUID[],                        -- related network_flows
    ml_model        VARCHAR(50),
    ai_narrative    TEXT,                           -- LLM-generated explanation
    ai_playbook     TEXT,                           -- LLM-generated response
steps
    resolved_at     TIMESTAMPTZ,
    resolved_by     UUID REFERENCES users(id),
    resolution_note TEXT,
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    updated_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Threat Intelligence

```
CREATE TABLE threat_intel_iocs (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    ioc_type        VARCHAR(20) NOT NULL,         -- ip, domain, url, hash, email
    ioc_value       VARCHAR(500) NOT NULL,
    threat_type     VARCHAR(100),                 -- malware, phishing, c2,
scanner, botnet
    severity        VARCHAR(20),
    source          VARCHAR(100) NOT NULL,        -- otx, abuseipdb, virustotal,
manual
    source_ref      VARCHAR(500),                 -- reference URL
    first_seen      TIMESTAMPTZ,
    last_seen       TIMESTAMPTZ,
    confidence      REAL,
    tags            TEXT[],
    raw_data        JSONB,
    is_active       BOOLEAN DEFAULT true,
    created_at      TIMESTAMPTZ DEFAULT NOW(),
```

```
    updated_at      TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(ioc_type, ioc_value, source)
);

CREATE INDEX idx_iocs_value ON threat_intel_iocs(ioc_value);
CREATE INDEX idx_iocs_type ON threat_intel_iocs(ioc_type);
```

ML Models Registry

```
CREATE TABLE ml_models (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name            VARCHAR(100) NOT NULL,         -- isolation_forest_v1,
random_forest_v2
    model_type      VARCHAR(50) NOT NULL,          -- isolation_forest,
random_forest, autoencoder
    version         VARCHAR(20) NOT NULL,
    status          VARCHAR(20) DEFAULT 'training', -- training, active, retired,
failed
    dataset         VARCHAR(100),                   -- NSL-KDD, CICIDS2017
    metrics         JSONB,                          -- {accuracy, precision,
recall, f1, auc, confusion_matrix}
    hyperparams     JSONB,                          -- training configuration
    file_path       VARCHAR(500),                   -- path to serialized model
    training_time   REAL,                           -- seconds
    inference_time  REAL,                           -- avg ms per prediction
    is_active       BOOLEAN DEFAULT false,          -- currently used for
inference
    trained_at      TIMESTAMPTZ,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Additional Tables

```
-- Capture sessions
CREATE TABLE capture_sessions (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    interface       VARCHAR(50),
    status          VARCHAR(20) DEFAULT 'running',  -- running, stopped, error
    packets_total   BIGINT DEFAULT 0,
    flows_total     BIGINT DEFAULT 0,
    anomalies_total INTEGER DEFAULT 0,
    started_at      TIMESTAMPTZ DEFAULT NOW(),
    stopped_at      TIMESTAMPTZ,
    config          JSONB                            -- BPF filter, timeout
settings
);

-- PCAP uploads for forensics
```

```sql
CREATE TABLE pcap_uploads (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    filename        VARCHAR(500) NOT NULL,
    file_size       BIGINT,
    file_path       VARCHAR(500),
    status          VARCHAR(20) DEFAULT 'pending',   -- pending, processing,
complete, error
    packets_count   INTEGER,
    flows_extracted INTEGER,
    anomalies_found INTEGER,
    uploaded_by     UUID REFERENCES users(id),
    processed_at    TIMESTAMPTZ,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- LLM conversation history
CREATE TABLE llm_conversations (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         UUID REFERENCES users(id),
    messages        JSONB NOT NULL,                  -- [{role, content,
timestamp}]
    context_type    VARCHAR(50),                     -- alert_analysis,
threat_hunt, report, general
    context_id      UUID,                            -- related alert/flow/report
ID
    tokens_used     INTEGER,
    cost_usd        REAL,
    provider        VARCHAR(50),                     -- deepseek, glm, groq
    created_at      TIMESTAMPTZ DEFAULT NOW(),
    updated_at      TIMESTAMPTZ DEFAULT NOW()
);

-- System configuration
CREATE TABLE system_config (
    key             VARCHAR(100) PRIMARY KEY,
    value           JSONB NOT NULL,
    description     TEXT,
    updated_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Audit log
CREATE TABLE audit_log (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         UUID REFERENCES users(id),
    action          VARCHAR(100) NOT NULL,
    entity_type     VARCHAR(50),
    entity_id       UUID,
    details         JSONB,
    ip_address      INET,
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

4.3 Entity Relationship Summary

```
users ──┬── alerts (assigned_to, resolved_by)
        ├── pcap_uploads (uploaded_by)
        ├── llm_conversations (user_id)
        └── audit_log (user_id)

network_flows ─── alerts (flow_ids[])
                  ├── anomaly scoring from ml_models
                  └── correlation with threat_intel_iocs

capture_sessions ─── network_flows (source context)

ml_models ─── network_flows (ml_model reference)
           └── alerts (ml_model reference)
```

---

5. API Architecture & Design

5.1 RESTful API Design

Base URL: `https://api.threatmatrix-ai.com/api/v1/`

Method	Endpoint	Description
Auth		
POST	`/auth/login`	JWT token pair (access + refresh)
POST	`/auth/register`	Create user (admin only)
POST	`/auth/refresh`	Refresh access token
GET	`/auth/me`	Current user profile
Flows		
GET	`/flows/`	List flows (paginated, filterable)
GET	`/flows/{id}`	Single flow detail
GET	`/flows/stats`	Aggregated flow statistics
GET	`/flows/top-talkers`	Top IPs by volume
GET	`/flows/protocols`	Protocol distribution
POST	`/flows/search`	Advanced flow search
Alerts		
GET	`/alerts/`	List alerts (filterable by severity/status)
GET	`/alerts/{id}`	Single alert with related flows

Method	Endpoint	Description
PATCH	`/alerts/{id}/status`	Update alert status
PATCH	`/alerts/{id}/assign`	Assign to analyst
GET	`/alerts/stats`	Alert statistics and trends
ML		
GET	`/ml/models`	List all trained models
GET	`/ml/models/{id}/metrics`	Model performance metrics
POST	`/ml/predict`	Manual prediction on feature vector
POST	`/ml/retrain`	Trigger model retraining
GET	`/ml/comparison`	Side-by-side model comparison
Intel		
GET	`/intel/iocs`	List IOCs (paginated)
GET	`/intel/lookup/{ip_or_domain}`	IP/domain reputation lookup
POST	`/intel/sync`	Trigger feed synchronization
GET	`/intel/feeds/status`	Feed health status
LLM		
POST	`/llm/chat`	Send message, get AI response (streaming)
POST	`/llm/analyze-alert/{id}`	Generate alert narrative
POST	`/llm/briefing`	Generate threat briefing
POST	`/llm/translate`	Translate text to Amharic/English
GET	`/llm/budget`	Token usage and budget status
Capture		
GET	`/capture/status`	Current capture session status
POST	`/capture/start`	Start packet capture
POST	`/capture/stop`	Stop packet capture
POST	`/capture/upload-pcap`	Upload PCAP for analysis
GET	`/capture/interfaces`	Available network interfaces
Reports		
POST	`/reports/generate`	Generate PDF report
GET	`/reports/`	List generated reports
GET	`/reports/{id}/download`	Download PDF

Method	Endpoint	Description
System		
GET	`/system/health`	Service health check
GET	`/system/metrics`	System performance metrics
GET	`/system/config`	System configuration

5.2 WebSocket Events

Endpoint: `wss://api.threatmatrix-ai.com/ws/`

Channel	Event	Payload	Direction
`flows:live`	`new_flow`	Flow record with features	Server → Client
`alerts:live`	`new_alert`	Alert object	Server → Client
`alerts:live`	`alert_updated`	Updated alert	Server → Client
`system:status`	`capture_status`	Engine status	Server → Client
`system:status`	`system_metrics`	CPU/mem/throughput	Server → Client
`ml:live`	`anomaly_detected`	Anomaly with score	Server → Client
`llm:stream`	`token`	Streaming LLM response	Server → Client

---

6. Real-Time Communication Strategy

6.1 Architecture: WebSocket + Redis Pub/Sub

```
Capture Engine ──► Redis Pub/Sub ──► FastAPI ──► WebSocket ──► Browser
                   (flows:live)      (subscriber)  (broadcast)  (N clients)

ML Worker ────► Redis Pub/Sub ──► FastAPI ──► WebSocket ──► Browser
                (alerts:live)     (subscriber)  (broadcast)
```

6.2 Why Redis Pub/Sub (Not Just WebSocket)

Concern	WebSocket Only	WebSocket + Redis
Multi-worker broadcasting	❌ Each worker has own connections	☑ Redis is central broker
Capture → Backend decoupling	❌ Direct coupling	☑ Capture publishes independently
Message durability	❌ Lost if no client connected	☑ Can add Redis Streams for replay

Concern	WebSocket Only	WebSocket + Redis
Rate limiting	Manual	Built-in Redis capabilities
LLM response caching	Separate system needed	☑ Redis cache built-in

6.3 Client-Side Strategy

```
Browser connects → WebSocket
  → Subscribes to: flows:live, alerts:live, system:status
  → Receives events → Updates React state via context/store
  → UI components re-render with new data
  → Reconnection logic with exponential backoff
```

---

7. Security Architecture

7.1 Authentication & Authorization

Layer	Implementation
Authentication	JWT (access token: 15 min, refresh: 7 days)
Password Storage	bcrypt with salt rounds=12
RBAC Roles	admin, soc_manager, analyst, viewer
API Protection	Bearer token required on all endpoints except `/auth/login`
WebSocket Auth	Token passed as query parameter on connection

7.2 Role Permissions Matrix

Permission	Admin	SOC Manager	Analyst	Viewer
View Dashboard	☑	☑	☑	☑
View Alerts	☑	☑	☑	☑
Acknowledge Alerts	☑	☑	☑	✕
Resolve Alerts	☑	☑	✕	✕
Use AI Analyst	☑	☑	☑	✕
Upload PCAP	☑	☑	☑	✕
Generate Reports	☑	☑	☑	✕
Manage Users	☑	✕	✕	✕
Start/Stop Capture	☑	☑	✕	✕
Retrain Models	☑	✕	✕	✕

Permission	Admin	SOC Manager	Analyst	Viewer
System Config	☑	✕	✕	✕
View LLM Budget	☑	☑	✕	✕

8. Technology Stack Decisions

8.1 Final Stack

Layer	Choice	Version	Rationale
Frontend Framework	Next.js	16.x	Team expertise proven via Pana ERP, SSR/SSG, API routes
Frontend Language	TypeScript	5.x (strict)	Type safety, DDD alignment
Styling	Vanilla CSS + CSS Variables	—	Maximum control for War Room glassmorphism effects
Maps	Deck.gl + Maplibre GL	Latest	Zero cost, WebGL-powered, open source
Charts	Recharts	Latest	React-native, composable, themeable
Real-time (client)	Native WebSocket API	—	No library needed, lightweight
Backend Framework	FastAPI	0.110+	Async, auto-docs, Python ML ecosystem
Backend Language	Python	3.11+	ML/security industry standard
Database	PostgreSQL	16	Enterprise-grade, JSONB, INET types, full-text search
Cache/Pub-Sub	Redis	7.x	Pub/Sub, caching, rate limiting
ML	scikit-learn + TensorFlow	Latest	Isolation Forest, Random Forest, Autoencoder
Packet Capture	Scapy	2.5+	Pure Python, no external deps
LLM Providers	DeepSeek, GLM, Groq	—	Cheapest quality ratio, open-source models
i18n	next-intl	Latest	Battle-tested Next.js internationalization
PDF Generation	ReportLab (backend)	—	Python-native PDF creation
Deployment	Docker Compose	V2	Single-command deployment
Frontend Hosting	Vercel	—	Free tier, automatic HTTPS, edge network
SSL	Let's Encrypt	—	Free TLS certificates

Layer	Choice	Version	Rationale
Icons	Lucide React	Latest	Consistent, lightweight icon set
Animations	Framer Motion	12.x	Micro-interactions, page transitions

9. Cost Infrastructure Analysis

9.1 Infrastructure Costs

Component	Provider	Monthly Cost	Notes
VPS	Already owned	$0	High-spec, guaranteed uptime
PostgreSQL	Self-hosted (Docker)	$0	On VPS
Redis	Self-hosted (Docker)	$0	On VPS
Frontend hosting	Vercel (free tier)	$0	100GB bandwidth
SSL certificate	Let's Encrypt	$0	Auto-renewal
Domain	Vercel subdomain	$0	threatmatrix-ai.vercel.app
Map tiles	Maplibre (OSM)	$0	Open-source tiles
Infrastructure Total		**$0/month**	

9.2 LLM API Budget ($100-200 total)

Provider	Model	Rate	Allocated	Expected Usage
DeepSeek	DeepSeek-V3	$0.14/M in, $0.28/M out	$40-60	Complex analysis, threat narratives
GLM (Zhipu)	GLM-4-Flash	$0.01/M tokens	$15-25	Bulk classification, translations
Groq	Llama 3.3 70B	$0.06/M tokens	$20-30	Real-time alert summaries (fastest)
Reserve	—	—	$30-50	Demo day buffer, unexpected spikes
LLM Total			**$100-165**	Covers full project + demo period

9.3 Budget Protection Mechanisms

Mechanism	Implementation
Token Budget Tracker	PostgreSQL table tracking daily/monthly spend per provider
Response Caching	Redis cache for identical/similar queries (TTL: 1 hour)
Rate Limiting	Max requests per user per minute via Redis

Mechanism	Implementation
Provider Fallback	If primary provider is expensive/down, fall to cheaper alternative
Alert Throttling	Don't re-analyze the same alert type within 5 minutes
Hard Budget Cap	System refuses LLM calls when monthly budget exhausted

---

> **End of Part 2** — Continue to Part 3: Module Specifications & UI/UX Design System

---

ThreatMatrix AI Master Documentation v1.0 — Part 2 of 5

© 2026 ThreatMatrix AI. All rights reserved.